

Lead Enrichment: Clay Billing, and the Bright Data + Claude Alternative

Cost breakdown and data-completeness findings for Project Beacon's LinkedIn enrichment pipeline

Prepared 2026-08-24 · Sources: clay.com official pricing/docs, Clay community & university docs, platform.claude.com pricing, and this repository's own captured Bright Data / Claude web-search test data

Contents

1. Executive summary
2. Finding: does Bright Data + Claude already deliver complete profiles?
3. Cost of the Claude + web_search path
4. Clay billing model in detail
5. Bring-your-own-key (BYOK) breakdown
6. Clay plan tiers
7. Recommendation & open item

1. Executive summary

Clay's manual proof-of-concept delivered complete LinkedIn profile data ("everything we want"), but at a cost the team flagged as too high. This report answers two questions raised in that discussion: (a) can the pipeline's existing Bright Data scrape, optionally supplemented by Claude's `web_search` tool, reach the same completeness for free/cheaper, and (b) exactly how is Clay billed, and does bringing your own LLM/data-provider API keys meaningfully reduce that bill.

Verdict: No — Bright Data's current scrape and Claude `web_search` are both structurally limited to shallow/snippet-level LinkedIn data, and cannot reliably reproduce what Clay's waterfall returned. Clay's cost is real, but it is paying for genuine data-source depth, not for something the existing stack can already do for free. Clay's own bring-your-own-key feature only reduces part of the bill, and only under specific conditions detailed in Section 5.

2. Finding: does Bright Data + Claude already deliver complete profiles?

CONFIRMED FROM REPO DATA

2.1 Bright Data's own raw scrape — checked directly against 10 real profiles

Rather than relying on documentation claims, a real captured Bright Data response batch already in the repo (`P0C's/linkedin_poc/raw_responses_LI3.json`) was inspected profile-by-profile for the two fields this report is about: the About section and the Experience (work history) list.

#	Profile	About — length	About — content	Experience
1	Tammy Pérez	86 chars	Truncated, ends "..."	null
2	Hani Almarzooq	86 chars	Truncated, ends "..."	null
3	Traduttori RH	68 chars	Truncated, ends "..."	null
4	Erika Weber-Guzman, Ed.D.	89 chars	Truncated, ends "..."	null
5	Hani Almarzooq (dup)	86 chars	Truncated, ends "..."	null
6	Cecilia de Tezanos	90 chars	Truncated, ends "..."	null
7	Linka M.	0	None (no public data)	null
8	Michelle Rosslee	1,276 chars	Full, multi-sentence	5 dated roles, full descriptions
9	Chloe Zhang	0	None (no public data)	null
10	(name missing)	0	None (no public data)	null

6 of 10 profiles returned an About field truncated to 86–90 characters, every one ending in an ellipsis — a systematic snippet limit in the dataset itself, not a parsing artifact. **9 of 10** returned `null` for Experience. Only **1 of 10** (Michelle Rosslee) came back genuinely complete. This means the pipeline's own parser can be working exactly as designed and still surface an incomplete profile most of the time — the ceiling is the data Bright Data's dataset captured, not the code reading it.

2.2 Claude `web_search` as a fallback — already tested in this repo, and unreliable

A prior experiment (`enrichment_pipeline/scratch_claude_websearch_linkedin.py`) tested whether Claude, using its `web_search` tool, could independently reconstruct a full profile (About, dated work history, headline, skills) for leads Bright Data scraped poorly. It ran against 3 real leads on Claude Opus 5, with up to 8 searches allowed per lead.

Lead	Outcome	Detail
Divya Shyam	Total failure	Every field null; <code>could_not_find_anything: true</code>
Mathumitha Senthil	Partial	About: null (none at all). Experience: 4 roles, but all with <code>dates: null</code> and single-sentence generic filler.
Alex Anthraper	Partial, self-flagged incomplete	Model's own words, quoted verbatim below.

"Experienced project manager and multilingual content specialist with 10+ years in media... (only this truncated preview of the About section is publicly visible; the complete text could not be retrieved)"
— Claude's own output for Alex Anthraper, About field

Why this fails: LinkedIn blocks Claude's `web_fetch` tool outright, so `web_search` only ever sees Google-indexed snippet fragments of a LinkedIn page — never the real page content. It hits the exact same shallow-data ceiling as Bright Data's own scrape, just from a different angle. 1 of 3 test leads returned nothing at all.

3. Cost of the Claude + `web_search` path

NOT RECOMMENDED — INCLUDED FOR COMPLETENESS

Cost component	Rate
Web search (server tool)	\$10.00 per 1,000 searches
Claude Opus 5 — input tokens	\$5.00 / MTok
Claude Opus 5 — output tokens	\$25.00 / MTok
Claude Sonnet 5 — input tokens	\$2.00 / MTok
Claude Sonnet 5 — output tokens	\$10.00 / MTok
Code execution tool	Free when combined with <code>web_search</code>

Actual measured usage from the 3-lead experiment (Claude Opus 5, 8 searches/lead, capped):

Lead	Input tokens	Output tokens	Searches	Est. cost
Divya Shyam (failed)	66,800	1,700	8	\$0.46
Mathumitha Senthil (partial)	120,900	3,000	8	\$0.76
Alex Anthraper (partial)	75,200	3,200	8	\$0.53

Even the highest-token run costs under \$1/lead — but every run in this sample was either a total miss or an incomplete, self-flagged-truncated result. Cost is not the problem with this path; reliability and completeness are. Swapping to Sonnet 5 would cut token cost roughly 3–4x further, but would not change the underlying data-access ceiling (LinkedIn blocking `web_fetch` applies regardless of model).

4. Clay billing model in detail

Clay bills on two independent meters. Understanding which one a given enrichment step draws from is the key to predicting cost and to knowing whether BYOK will actually help.

	Data Credits	Actions
What it pays for	Each lookup against a third-party data provider in a waterfall (Apollo, Hunter, Findymail, Clearbit, etc.) — one credit per provider <i>attempted</i> , hit or miss.	Platform operations: running an enrichment step, an AI/LLM prompt call, a CRM push, an API request.
Baseline unit cost	~\$0.05 each (Clay states frequently-used enrichments run cheaper via negotiated volume rates — more like \$0.02–0.05 in practice)	<\$0.01 each
Failed lookups	Still billed. A 5-provider waterfall where the first 4 miss and the 5th hits still consumes 5 credits, not 1.	

This is why a single "complete profile" waterfall column can be expensive even when it works: every provider it tries along the way before finding data is a paid attempt, win or lose.

5. Bring-your-own-key (BYOK) breakdown

Clay explicitly supports connecting your own API keys instead of its managed ones (Settings > Connections, or per-column in the Enrichment Panel). The effect is different depending on which kind of column it's applied to.

5.1 AI / LLM columns (OpenAI, Anthropic supported)

	Clay's managed key	Your own key
Per-call AI cost	~\$0.01 / call	~\$0.003 / call (OpenAI, per Clay's own FAQ)
Actions charge	Still applies either way — BYOK does not remove this	
Trade-off	Faster, per Clay's own guidance	Cheaper, per Clay's own guidance

Clay's official framing, quoted directly: "It's cheaper per call to use your own OpenAI API key, but faster to go through ours."

This only meaningfully reduces cost if the expensive part of a Clay run was an **AI/Claygent research column** (a prompt-driven research agent), not the core contact-finding waterfall.

5.2 Data-provider columns (Apollo, Findymail, Prospeo, Debounce, NeverBounce confirmed)

	Clay's managed key	Your own key
Billing	Clay Data Credit charged per attempt	Clay's Data Credit charge is bypassed — the provider bills your own account directly instead
Reported savings	~70–80% vs. Clay's built-in rate, at high volume	
Plan gating	Some providers require a paid tier — e.g. Findymail BYOK requires Clay Starter (\$149/mo) minimum , not available on Free	
Failed lookups	Still cost you, regardless of whose key is used	

The catch that matters most: this only saves money if you already have — or are willing to pay for — your own separate subscription with that specific provider (Apollo, Findymail, etc.). Otherwise BYOK just moves the identical cost onto a different invoice; it does not make the underlying data lookup free.

6. Clay plan tiers

Plan	Price / month	Data Credits / month	Actions / month
Free	\$0	100	500
Launch	\$167 (annual) / ~\$185 (monthly)	3,000	15,000
Growth	\$446 (annual) / ~\$495 (monthly)	6,000	40,000
Enterprise	Custom	Custom	Custom

Source: clay.com/pricing, fetched directly. Annual billing carries a 10% discount over monthly.

7. Recommendation & open item

Bright Data / Claude web_search is not a substitute for Clay for complete About/Experience data — both are capped by the same shallow public-page access problem, confirmed against this project's own captured data, not assumed from documentation.

Clay's cost is largely unavoidable for the completeness it delivers — BYOK helps only in two narrow cases: (1) if the spend was driven by an AI/Claygent research column, or (2) if you already hold (or will buy) your own subscription with the specific waterfall provider Clay used. It does not create savings out of nothing, and failed-lookup billing applies regardless of key ownership.

Open item: the exact \$/lead figure for your Clay POC run still depends on which columns you actually enrich with (Email/Phone Finder waterfall vs. an AI/Claygent column) and what Clay's own credit-usage panel showed for that run. That number, once available, converts this framework into a precise cost projection at your real monthly LinkedIn volume.

Sources: clay.com/pricing, clay.com/waterfall-enrichment, clay.com/faq (OpenAI key guidance), university.clay.com (AI in Clay; credit conservation guide), community.clay.com, platform.claude.com/docs (pricing, web search tool, code execution tool) — all fetched 2026-08-24. Bright Data and Claude web_search findings drawn directly from this repository:

`POC's/linkedin_poc/raw_responses_LI3.json` and
`enrichment_pipeline/output/claude_websearch_formatted_output.json` /
`scratch_claude_websearch_linkedin.py`.